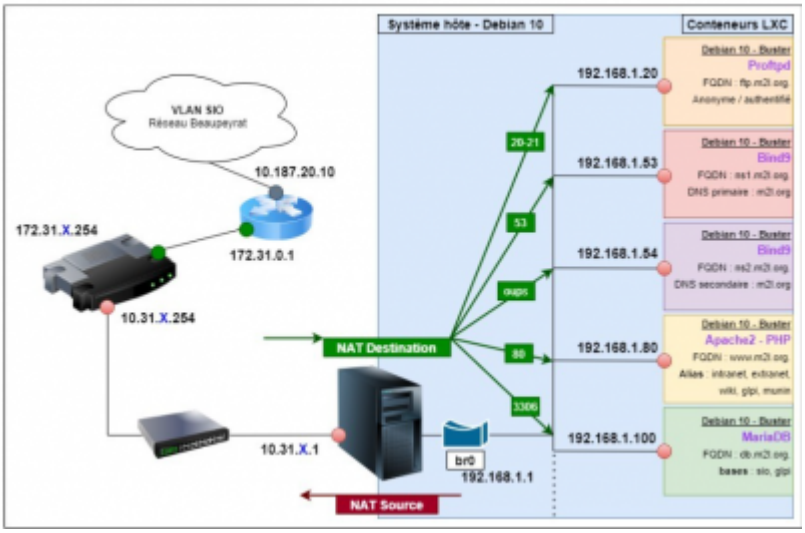
Conteneurs LXC

Quelques notions:

- **Un conteneur :** c'est une boite virtuelle isolée pour installer des services (des applications avec tous les éléments dont elles ont besoin pour fonctionner).
- **Un pont :** c'est un outil informatique qui nous permet de connecter deux réseaux locaux utilisant le même protocole.
- **bridge-utils:** c'est un paquet qui contient un utilitaire nécessaire pour créer et gérer un périphérique de pont.

Mise en place de conteneurs LXC

La mise en place de conteneur LXC (LinuX Container) peut se faire de plusieurs manières. Ici on s'intéressera à la méthode qui consiste à placer les conteneurs sur le même réseau publique que notre hôte(10.31.48.1/20) et créer un pont qu'on appellera br0.



Remarque : Dans notre configuration, l'adresse du réseau sera 10.31.48.1/20.

1. Mise à jour et installation de LXC

Pour pouvoir installer LXC, on met à jour la liste des paquets et on compare les versions de logiciels dans la machine à celui des versions dans les dépôts.

```
host# apt-get update        ← comparaison des versions des logiciels
disponibles
host# apt-get upgrade       ← mise à jour des paquets
host# apt-get install lxc   ← installation de LXC
```

Après cela, on vérifie la configuration de notre lxc :

```
host# lxc-checkconfig
```

Il faut s'assurer de taper les commandes là où il faut. En effet il faut faire la différence entre les commandes à taper dans le terminal de l'hôte (root@srv#

lxc-...) et les commandes à taper dans le terminal des conteneurs (#web ou #template).

création et configuration du pont br0

Pour créer le pont br0, on installe d'abord bridge-utils et là on peut le créer:

```
root@srv-g3:~# apt-get install bridge-utils   ← installation de bridge-utils
root@srv-g3:~# brctl addbr br0                ← "addbr" pour ajouter le pont
```

Maintenant on peut passer à la configuration de notre pont br0 en tapant quelques lignes de commandes soit dans le fichier rc.local tout en veillant que l'ordre d'exécution des commandes soit bien respecté soit dans le terminal du serveur.

Dans notre cas, on s'intéressera au fichier rc.local.

```
#!/bin/sh -e
ifconfig eno1 0.0.0.0

brctl addbr br0
ifconfig br0 10.31.48.1/20 up
brctl addif br0 eno1

route add default gw 10.31.63.254
echo "nameserver 8.8.8.8" > /etc/resolv.conf
```

On pourrait vérifier la configuration du pont pour pouvoir par la suite configurer par défaut les conteneurs et s'assurer que notre interface eno1 n'a plus d'adresse puisqu'on a attribué son adresse réseau au pont br0.

```
root@srv-g3:/etc# ifconfig
br0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.31.48.1  netmask 255.255.240.0  broadcast 10.31.63.255
        inet6 fe80::94ab:a1ff:fe11:42e2  prefixlen 64  scopeid 0x20<link>
        ether 96:ab:a1:11:42:e2  txqueuelen 1000  (Ethernet)
        RX packets 91285  bytes 73640244 (70.2 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 63152  bytes 5536351 (5.2 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

eno1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet6 fe80::9640:c9ff:fe4a:1c44  prefixlen 64  scopeid 0x20<link>
        ether 94:40:c9:4a:1c:44  txqueuelen 1000  (Ethernet)
        RX packets 123984  bytes 114510686 (109.2 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 94307  bytes 12082233 (11.5 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 17
```

```
root@srv-g3:/etc# root@srv-g3:/etc# brctl show
bridge name     bridge id               STP enabled     interfaces
br0             8000.96aba11142e2       no              eno1
lxcbr0          8000.00163e000000       no
```

Création et configuration des conteneurs

Cela implique d'abord la configuration par défaut du premier conteneur appelé template dans le fichier default.conf (**“nano /etc/lxc/default.conf”**)

```
lxc.net.0.type = veth
lxc.net.0.link = br0
```

```
lxc.net.0.flags = up
lxc.net.0.name = eth0
lxc.apparmor.profile = generated
lxc.apparmor.allow_nesting = 1
```

Ensuite on tape la commande ci dessous pour créer le conteneur template.

```
root@srv-g3:~# lxc-create -n template -t debian -- -r bookworm
```

Après avoir créé le conteneur template, on va le démarrer grâce à la commande lxc-start template. Cependant cette méthode n'implique pas le démarrage automatique à chaque fois et pour se faire, on ajoute ***"lxc.start.auto = 1"*** dans le fichier config du conteneur dans le répertoire */var/lib/lxc/template.*

- Manipulation du conteneur

Pour pouvoir manipuler notre conteneur, on lance son terminal avec la commande lxc-attach template et on commence à le configurer pour pouvoir installer et mettre à jour les outils de base. Pour la configuration, on peut soit taper ces lignes de commandes direct sur le terminal de notre conteneur, soit on le configure direct dans le fichier "interfaces" qui est dans le répertoire **/etc/network**

#configuration du conteneur par ligne de commandes

```
 #configuration IP
template#ifconfig eth0 10.31.48.2 up
template#route add default gw 10.31.63.254
 #mise à jour et installation des outils de base
template#apt update
template#apt upgrade
template#apt install sudo net-tools tcpdump nano iputils-ping dbus
```

#configuration du conteneur dans le fichier /etc/network/interfaces

```
web# cat /etc/network/interfaces
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet static
        address 10.31.X.80/20
        gateway 10.31.X.254
        dns-nameservers 8.8.8.8
```

NB : nous allons ajouter à notre conteneur *template* le module **rsyslog** (qui fournit une installation pour exécuter un serveur d'enregistrement et pour configurer des systèmes individuels pour qu'ils envoient leurs fichiers journaux sur le serveur d'enregistrement)

```
apt update
apt install rsyslog
```

Le procédé sera le même pour les autres conteneurs sauf qu'au lieu de les créer, on copie juste la configuration du conteneur de base(template) et on y apporte des modifications dans le fichier interfaces.

```
#copie du conteneur
root@srv-g3:~#lxc-copy -n template -N  web
#renommer le conteneur
root@srv-g3:~#hostnamectl set-hostname web
```

communication via ssh

Cela implique de générer des paires de clés et de placer la clé publique dans le fichier authorized_keys. Vue qu'on avait déjà réalisé ce même processus dans le TP précédent, on copie juste nos clés publiques qu'on avait déposé dans le fichier authorized_keys du serveur(dans le répertoire /.ssh/authorized_keys) dans le fichier auhtorized_keys situé dans .ssh qu'on a créé pour le conteneur.

Annexe

Quelques commandes utiles

```
• lxc-start container                          → pour démarrer le conteneur
• lxc-stop container                           → pour arrêter le conteneur
• lxc-copy <container -N new_container         → pour copier le conteneur
• lxc-destroy container                        → pour supprimer le conteneur
• lxc-checkconfig                              → pour vérifier la
configuration du conteneur
• lxc-attach container                         → pour accéder au conteneur
• lxc-info [-n] <container>                    → pour visualiser les infos
concernant le conteneur
• lxc-ls                                       → pour voir la liste des
conteneurs disponibles
```

Ces commandes ci-dessus se tapent sur le terminal du serveur et non celui des conteneurs.

POUR CHANGER L'IP DU CONTENEUR, IL FAUT SE RENDRE DANS */etc/network/interfaces*, PUIS *systemctl restart networking*.

From:
https://sisr2.beaupeyrat.com/ - **Documentations SIO2 option SISR**

Permanent link:
https://sisr2.beaupeyrat.com/doku.php?id=sisr1-g3:mission1.2

Last update: **2024/02/14 17:58**